



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2233 Programación Avanzada (2026-2)

Ejercicios propuestos 2

Programación funcional, threading, serialización y networking

■ Entrega

- **Fecha y hora límite:** 16 de octubre de 2026, hasta las 20:00 horas.
- **Lugar:** Repositorio personal de GitHub, en la carpeta EP2/. El código debe encontrarse en la rama (*branch*) por defecto del repositorio: `main`.
- **Contacto:** Para informar que los entregables se encuentran disponibles y solicitar feedback, debes comunicarte con el/la profesor/a de tu respectiva sección. No se aceptarán entregables enviados después de las **20:00 horas del 16 de octubre de 2026**.
- **Condiciones de revisión:** Solo se entregará feedback a las entregas que cumplan con las condiciones de ejecución, formato, legibilidad y organización indicadas en la [sección de Restricciones y alcances](#).

Objetivos

- Aplicar el paradigma de programación funcional (generadores, `map`, `filter`, `reduce`, `lambda`, `itertools` y `collections`) para cargar y consultar datos sin materializar estructuras intermedias.
- Modelar un problema concurrente con `threads`, decidiendo qué recursos se comparten y cómo sincronizarlos (`Lock`, `Event`, `Timer`, `queue.Queue`).
- Diseñar e implementar una arquitectura cliente-servidor con `sockets` TCP, donde el servidor atiende varios clientes a la vez y sobrevive a desconexiones.
- Serializar objetos con `pickle` y con `json`, personalizando la serialización cuando el objeto no puede o no debe viajar tal cual.
- Implementar un protocolo de comunicación propio a nivel de `bytes`.
- Exponer y consumir un servicio web simple con Flask y `requests`.

Índice

1. Introducción	3
2. La casa de remates	3
2.1. Reglas de una puja	3
2.2. Extensión del lote	4
2.3. Cierre de un lote	4
2.4. Ranking de la sesión	4
3. Cliente	4
3.1. Lo que el cliente debe cumplir	5
4. Servidor	5
4.1. Lo que el servidor debe cumplir	6
4.2. Logs del servidor	7
5. Programación funcional	7
5.1. Carga de datos	7
5.2. Consultas sobre el catálogo	8
5.3. Consultas sobre el historial	8
5.4. Dónde se usa cada consulta	8
6. Protocolo de comunicación	9
6.1. Mensajes	9
6.2. Codificación de bytes	10
7. Serialización	11
7.1. pickle y modelos.py	11
7.2. JSON	12
8. WebServices	12
9. Archivos del programa	13
9.1. Estructura de la entrega	13
9.2. Archivos CSV	13
9.2.1. lotes.csv	14
9.2.2. postores.csv	14
9.2.3. historial.csv	14
9.3. .gitignore	14
10. Parámetros	14
11. Pruebas	15
12. Restricciones y alcances	15

1. Introducción

Tras años acumulando objetos en las bodegas del DCC (servidores que nunca prendieron, un reloj cucú sin cucú, la silla ergonómica de un profesor que juró que “era cómoda”), la administración decidió que la única forma de deshacerse de todo con dignidad era rematarlo. Así nace *DCCubasta*, la primera casa de remates del departamento, donde cada lote se subasta al mejor postor y donde el martillo lo maneja un servidor que no acepta reclamos. Los postores se conectan desde sus consolas, pujan en tiempo real con sus preciados *DCCoins* y, cada vez que alguien ofrece en el último segundo, el subastador extiende el remate para que nadie diga que no tuvo oportunidad.

En este ejercicio propuesto desarrollarás el sistema completo: un **servidor** que administra el catálogo, recibe pujas de muchos postores a la vez sin que se pisen entre ellas, cierra cada lote a tiempo y guarda el historial; un **cliente** por consola desde el cual los postores pujan y consultan sus adjudicaciones; una pequeña **API web** para revisar el catálogo y el ranking sin conectarse al remate; y, como la administración desconfía de todos, un **protocolo de bytes cifrado** para que nadie espíe las pujas.

El enunciado describe **qué** debe hacer el programa y **qué no puede pasar**, y fija los nombres de archivos, comandos y funciones que necesitan ser comunes para poder revisar tu trabajo. **Cómo** organizar las clases, los threads, los mecanismos de sincronización y los módulos es parte del ejercicio y queda a tu criterio.

2. La casa de remates

El servidor administra un catálogo de **lotes** y un conjunto de **postores** registrados. Cada ejecución del servidor es un **remate**: se subastan uno a uno los primeros *LOTES_POR_REMATE* lotes del catálogo (en orden de *id_lote*) que no hayan sido adjudicados en ejecuciones anteriores. El remate comienza cuando hay *MINIMO_POSTORES* postores conectados y termina cuando se cierra el último lote. Si no queda ningún lote por subastar, el remate termina apenas comienza.

Cada postor tiene un **saldo** de DCCoins (su saldo inicial viene de *postores.csv*) que se descuenta solo cuando gana un lote.

2.1. Reglas de una puja

Cuando un postor envía una puja por un **monto**, el servidor evalúa las siguientes condiciones **en este orden**; la primera que falla determina el **motivo** del rechazo:

1. **no_hay_lote_abierto**: no hay ningún lote en subasta (antes de iniciar, entre lotes o después de terminar).
2. **monto_invalido**: *monto* no es un *int* o es menor o igual a 0.
3. **ya_lidera**: el postor ya es el líder del lote.
4. **monto_insuficiente**: si el lote no tiene pujas, el monto debe ser al menos *precio_base*; si tiene, al menos *precio_actual* + *INCREMENTO_MINIMO*.
5. **saldo_insuficiente**: $\text{saldo} < \text{monto} + \text{comision}(\text{monto})$, donde $\text{comision}(x) = \text{int}(x * \text{COMISION_CASA})$.

Si pasa todas las condiciones, la puja se acepta: el precio actual pasa a ser **monto**, el postor pasa a ser líder, se registra la puja y se informa a **todos** los conectados.

2.2. Extensión del lote

Si una puja es aceptada cuando quedan **menos de** `UMBRAL_EXTENSION` segundos y el lote lleva menos de `MAXIMO_EXTENSIONES` extensiones, el cierre del lote se reprograma para `EXTENSION_LOTE` segundos después de esa puja (el tiempo restante se **reemplaza**, no se suma), se incrementa el contador de extensiones y se informa a todos los conectados.

2.3. Cierre de un lote

Un lote se cierra exactamente cuando vence su tiempo, sin intervención de ningún postor. Si hay líder: `precio_final = precio_actual`, `comision = int(precio_final * COMISION_CASA)`, se descuenta `precio_final + comision` del saldo del líder, se crea una adjudicación y se guarda en `data/remates.json`. Si no hubo pujas, el lote se cierra sin ganador. En ambos casos se informa a todos los conectados y, tras `TIEMPO_ENTRE_LOTES` segundos, se abre el siguiente lote.

Las pujas son **vinculantes**: si el líder se desconectó antes del cierre, igual se le adjudica el lote y se le descuenta el saldo.

2.4. Ranking de la sesión

Al terminar el remate se informa a todos los conectados una lista de tuplas (`usuario`, `lotes_adjudicados`, `total_gastado`) ordenada por total gastado descendente, considerando solo las adjudicaciones de esta ejecución. En caso de empate se acepta cualquier orden.

No hay aleatoriedad en ninguna parte del programa.

3. Cliente

El cliente se ejecuta ubicando la terminal dentro de `cliente/` mediante:

```
python3 main.py
```

Al iniciar, lee `conexion.json` y se conecta al servidor. Si no puede, imprime `No fue posible conectarse al servidor en <host>:<puerto>` y termina. Luego pide el nombre de usuario por consola y lo envía al servidor; si es rechazado, muestra el motivo y vuelve a pedirlo, hasta `INTENTOS_INGRESO` veces; si se agotan, termina.

Una vez dentro, debe mostrar un menú similar al siguiente:

```
===== DCCubasta - Postor: rmunoz | Saldo: 12500 DCCoins =====
[1] Ver lote en subasta
[2] Pujar por el lote en subasta
[3] Mis adjudicaciones de esta sesión
[4] Catálogo de lotes (API)
[5] Detalle de un lote (API)
[6] Ranking histórico de postores (API)
[7] Salir
Opción:
```

Puedes modificar los textos y la presentación, pero todas las opciones indicadas deben estar disponibles.

- **Ver lote en subasta:** muestra id, nombre, categoría, año, precio base, precio actual, líder, cantidad de pujas, extensiones usadas y segundos restantes. Si no hay lote abierto, lo indica.

- **Pujar:** pide un monto por consola. Si no es un entero positivo, imprime un mensaje y vuelve al menú **sin** consultar al servidor. Si lo es, envía la puja y muestra si fue aceptada o el motivo del rechazo, en palabras.
- **Mis adjudicaciones:** lista los lotes ganados en esta sesión (nombre, precio final, comisión) y el saldo restante.
- **Catálogo de lotes:** pide una categoría (Enter para todas) y consulta `GET /lotes`. Muestra id, nombre, categoría, precio base y estado de cada lote, más el valor base total por categoría.
- **Detalle de un lote:** pide un id y consulta `GET /lotes/<id>`. Si la API responde 404, imprime El lote `<id>` no existe.
- **Ranking histórico:** consulta `GET /ranking?n=<TOP_RANKING>` y muestra la posición, el usuario, los lotes adjudicados y el total gastado de los primeros `TOP_RANKING` postores.
- **Salir:** avisa al servidor, cierra la conexión y termina.

3.1. Lo que el cliente debe cumplir

- Los **eventos** que envía el servidor (se abrió un lote, alguien pujó, el lote se extendió o cerró, terminó el remate) deben aparecer en pantalla **apenas llegan**, aunque el usuario esté en medio de escribir una opción o un monto. Se imprimen en una línea nueva, precedidos por `»`. No se exige que la consola quede “bonita”.
- Ninguna acción del menú puede dejar al programa esperando para siempre. Si el servidor no responde a un comando en `TIMEOUT_RESPUESTA` segundos, el cliente imprime El servidor no respondió a tiempo. y vuelve al menú.
- Si la API no responde, las opciones 4, 5 y 6 imprimen La API no está disponible. y vuelven al menú.
- Si el servidor se cae o avisa que se cierra, el cliente imprime `» Conexión con el servidor finalizada: <motivo>` y termina limpiamente, **sin mostrar un traceback**. Está permitido que lo haga recién cuando el usuario vuelva al menú (después de presionar Enter), porque `input()` es bloqueante.
- El cliente no debe conocer las pujas individuales de los demás postores: solo el líder, el precio actual y la cantidad de pujas.

4. Servidor

El servidor se ejecuta ubicando la terminal dentro de `servidor/` mediante `python3 main.py`. Debe:

1. Leer `conexion.json` (host, puerto TCP y puerto de la API) y `parametros.py`.
2. Cargar `data/lotes.csv`, `data/postores.csv` y `data/historial.csv` usando las funciones de `consultas.py` (sección 5), y `data/remates.json` (si no existe, lo crea vacío).
3. Imprimir el **resumen de la casa** (sección 5.4).
4. Dejar disponible la API (sección 8) y el socket TCP.
5. Aceptar postores mientras esté en ejecución (antes y durante el remate), hasta `MAXIMO_POSTORES` conectados a la vez. El primer mensaje de cada cliente debe ser `INGRESAR`.

6. Iniciar el remate cuando haya `MINIMO_POSTORES` postores y subastar los lotes según la sección 2. Los postores que se conectan después de iniciado participan desde el lote que esté abierto en ese momento.
7. Al cerrarse el último lote: difundir el ranking, avisar que se va a cerrar, cerrar todas las conexiones y **terminar por sí solo**.

4.1. Lo que el servidor debe cumplir

Estas son las propiedades que se verificarán ejecutando tu servidor con varios clientes a la vez. Cómo las garantizas (qué `threads` creas, qué recursos compartes y con qué mecanismo los proteges) es tu decisión, pero debes poder explicarla en el `README.md`.

- **Atención simultánea.** El servidor atiende a todos los postores conectados al mismo tiempo. Un cliente lento, bloqueado o que nunca envía nada no puede impedir que los demás pujen ni que el remate avance.
- **Una sola puja gana.** Si dos o más postores pujan por el mismo monto en el mismo instante, exactamente una puja se acepta y las demás se rechazan con `monto_insuficiente`. Nunca puede haber dos líderes, un precio actual inconsistente con el líder ni un saldo descontado dos veces.
- **El cierre es puntual.** Cada lote se cierra cuando vence su tiempo aunque nadie envíe mensajes. Una extensión invalida el cierre que estaba programado: un lote extendido no puede cerrarse antes de su nuevo plazo. Ningún cliente debe quedar esperando por culpa del cierre.
- **Los mensajes llegan enteros.** Todo evento se difunde a todos los conectados, y cada cliente recibe cada mensaje completo y en orden: los bytes de dos mensajes distintos nunca se entremezclan, aunque se generen al mismo tiempo desde distintas partes del servidor.
- **Consistencia con la API.** Lo que entrega la API (estado de un lote, precio actual, líder, ranking) debe corresponder al estado real del remate en ese instante: un lote no puede aparecer a la vez como adjudicado y en subasta.
- **Sin bloqueos ni espera activa.** El servidor nunca queda “pegado” (`deadlock`). No se permite esperar a que ocurra algo consultando repetidamente una condición en un ciclo con `sleep` (`busy waiting`): para eso están los mecanismos de sincronización de `threading` y `queue`. Una pausa de duración fija, como la de `TIEMPO_ENTRE_LOTES`, no es espera activa.
- **Nunca se cae por un cliente.** Un cliente que cierra la ventana, presiona Ctrl+C o pierde la red no afecta al resto: el servidor registra el `log` correspondiente, libera los recursos asociados a ese cliente y sigue funcionando. El usuario puede volver a ingresar. Si era el líder del lote, su puja sigue vigente.
- **Mensajes corruptos.** Si llega algo que no corresponde a un mensaje válido del protocolo (bloque fuera de secuencia, largo inconsistente, contenido que no se puede deserializar) y la conexión sigue viva, el servidor registra el `log`, responde `ERROR` con motivo `mensaje_corrupto` y sigue atendiendo a ese cliente.
- **Cierre ordenado.** Al terminar el remate (o si se interrumpe con Ctrl+C), el servidor avisa a los clientes con `SERVIDOR_CERRANDO` cuando puede, cierra sus conexiones y termina: el proceso no debe quedar vivo esperando a ningún `thread`.

4.2. Logs del servidor

El servidor debe imprimir, con este formato, al menos los siguientes mensajes:

```
[LOG] Servidor escuchando en <host>:<puerto>
[LOG] API disponible en http://<host>:<puerto_api>
[LOG] Conexión aceptada desde <ip>:<puerto_remoto>
[LOG] Postor <usuario> ingresó (saldo: <saldo>). Conectados: <n>/<MAXIMO_POSTORES>
[LOG] Ingreso rechazado desde <ip>:<puerto_remoto>: <motivo>
[LOG] Remate iniciado con <n> postores.
[LOG] Lote <id> '<nombre>' abierto. Precio base: <precio>. Cierra en <DURACION_LOTE> s.
[LOG] Puja aceptada: <usuario> ofrece <monto> por lote <id>.
[LOG] Puja rechazada de <usuario> por lote <id>: <motivo>.
[LOG] Lote <id> extendido (<k>/<MAXIMO_EXTENSIONES>): +<EXTENSION_LOTE> s.
[LOG] Lote <id> adjudicado a <usuario> por <precio> (comisión <comision>).
[LOG] Lote <id> cerrado sin ofertas.
[LOG] Postor <usuario> se desconectó.
[LOG] Postor <usuario> sufrió una desconexión inesperada.
[LOG] Mensaje corrupto recibido de <usuario>; se ignora.
[LOG] Remate terminado. Historial guardado en data/remates.json.
```

5. Programación funcional

El módulo `servidor/consultas.py` contiene toda la carga de datos y las consultas, y debe escribirse **exclusivamente con programación funcional**:

- Está **prohibido**: usar `for` o `while` fuera de funciones generadoras, expresiones generadoras y comprensiones; usar `list()`, `tuple()`, `set()` o `dict()` para materializar un iterable; llamar a `.append`, `.extend`, `.sort` o `sorted`; usar variables globales; y mutar los argumentos recibidos.
- Está **permitido**: `yield`, `yield from`, expresiones generadoras, comprensiones, `map`, `filter`, `functools.reduce`, `lambda`, `zip`, `enumerate`, `next`, `sum`, `min`, `max`, `any`, `all`, `itertools`, `collections.namedtuple`, `Counter`, `defaultdict`, `deque` y la sentencia `if`.
- Toda consulta debe funcionar recibiendo **generadores que se agotan**: cada argumento se recorre **una sola vez**.
- Las funciones de carga deben ser **generadoras**: no pueden leer el archivo completo en memoria.

La restricción aplica solo a `consultas.py`; en el resto del programa puedes programar como prefieras. Las firmas de las funciones en el archivo `consultas.py` deben respetarse tal cual están definidas.

5.1. Carga de datos

Función	Descripción
<code>cargar_lotes(ruta: str) -> Generator[Lote]</code>	Entrega un <code>Lote</code> por línea del archivo, con los tipos convertidos.
<code>cargar_postores(ruta: str) -> Generator[Postor]</code>	Ídem.
<code>cargar_historial(ruta: str) -> Generator[Registro]</code>	Ídem.

5.2. Consultas sobre el catálogo

Función	Descripción
<code>lotes_por_categoria(lotes, categoria: str None) -> Iterator[Lote]</code>	Si <code>categoria</code> es <code>None</code> entrega todos los lotes; si no, solo los de esa categoría.
<code>lote_por_id(lotes, id_lote: int) -> Lote None</code>	El lote con ese id, o <code>None</code> .
<code>valor_base_por_categoria(lotes) -> dict[str, int]</code>	Suma del <code>precio_base</code> de todos los lotes por categoría.
<code>lotes_mas_antiguos(lotes) -> tuple[Lote, ...]</code>	El o los lotes con el menor <code>anio</code> (año). En caso de empate, todos .

5.3. Consultas sobre el historial

Estas funciones reciben `registros` que pueden ser `Registro` (del CSV) o adjudicaciones de la sesión (sección 7.1): ambos tienen los atributos `usuario`, `id_lote` y `precio_final`, y tus funciones deben servir para los dos.

Función	Descripción
<code>adjudicaciones_de(registros, usuario: str) -> Iterator</code>	Registros del usuario.
<code>total_gastado_por_postor(registros) -> Counter</code>	<code>Counter</code> usuario $\rightarrow$ suma de <code>precio_final</code> .
<code>ranking_postores(registros, cantidad: int None) -> Iterator[tuple[str, int, int]]</code>	Tuplas <code>(usuario, lotes_adjudicados, total_gastado)</code> ordenadas por total descendente; las <code>cantidad</code> primeras, o todas si es <code>None</code> .
<code>adjudicaciones_por_categoria(registros, lotes) -> Counter</code>	Cantidad de registros por categoría del lote.
<code>sobrepromedio(registros, lotes) -> float</code>	Promedio de $(\text{precio_final} - \text{precio_base}) / \text{precio_base}$. 0.0 si no hay registros.
<code>categoria_favorita_por_region(registros, postores, lotes) -> dict[str, tuple[str, ...]]</code>	Para cada región, la(s) categoría(s) con más registros adjudicados a postores de esa región. En empate, todas.

5.4. Dónde se usa cada consulta

Ninguna consulta es decorativa:

- **Resumen de la casa** (al iniciar el servidor): se imprimen `valor_base_por_categoria`, `lotes_mas_antiguos`, `sobrepromedio` y `categoria_favorita_por_region`, cada una recibiendo generadores recién creados con las funciones de carga. Con los datos entregados, los resultados son:

Consulta	Resultado esperado
Valor base por categoría	Antigüedades 151900, Arte 94900, Coleccionables 137300, Rarezas 100100, Tecnología 164000
Lotes más antiguos	Año 1864: lotes con id 14, 34 y 49 (empate triple)
Sobrepromedio	63,0%
Categoría favorita por región	Centro: Antigüedades; Insular: Antigüedades; Norte: Arte; Sur: Antigüedades y Arte (empate)

- **Socket:** lote_por_id (apertura de lotes), adjudicaciones_de (MIS_ADJUDICACIONES) y ranking_postores (REMATE_TERMINADO).
- **API:** lotes_por_categoria, lote_por_id, valor_base_por_categoria, ranking_postores y adjudicaciones_por_categoria.

6. Protocolo de comunicación

6.1. Mensajes

Todo mensaje es un **diccionario** con la clave "comando". Los que envía el servidor llevan además "tipo": "respuesta" si contestan a un comando del cliente, o "evento" si los origina el servidor y se difunden a todos los conectados. El diccionario se serializa con `pickle` y se codifica según la sección 6.2.

Cliente → Servidor

Mensaje	Comentario
<code>{"comando": "INGRESAR", "usuario": str}</code>	Obligatoriamente el primero. Cualquier otro comando antes de ingresar recibe <code>ERROR</code> con motivo <code>no_ingresado</code> .
<code>{"comando": "ESTADO_LOTE"}</code>	
<code>{"comando": "PUJAR", "monto": int}</code>	Siempre sobre el lote abierto.
<code>{"comando": "MIS_ADJUDICACIONES"}</code>	
<code>{"comando": "SALIR"}</code>	El servidor no responde y cierra la conexión.

Servidor → Cliente, respuestas ("tipo": "respuesta")

Mensaje	Comentario
<code>{"comando": "INGRESAR", "exito": True, "saldo": int, "remate_iniciado": bool}</code>	
<code>{"comando": "INGRESAR", "exito": False, "motivo": str}</code>	Motivos: <code>usuario_inexistente</code> , <code>usuario_conectado</code> , <code>sala_llena</code> , <code>remate_terminado</code> .
<code>{"comando": "ESTADO_LOTE", "lote": LoteEnSubasta None, "saldo": int}</code>	<code>lote</code> viaja como objeto (sección 7.1); <code>None</code> si no hay lote abierto.
<code>{"comando": "PUJAR", "exito": True, "id_lote": int, "precio_actual": int, "extendido": bool, "tiempo_restante": float}</code>	
<code>{"comando": "PUJAR", "exito": False, "motivo": str}</code>	Motivos de la sección 2.1.
<code>{"comando": "MIS_ADJUDICACIONES", "adjudicaciones": list[Adjudicacion], "saldo": int}</code>	
<code>{"comando": "ERROR", "motivo": str}</code>	Motivos: <code>mensaje_corrupto</code> , <code>comando_desconocido</code> , <code>no_ingresado</code> .

Servidor → **Cliente**, eventos ("tipo": "evento", difundidos a todos)

```
{"comando": "REMATE_INICIADO", "postores": int, "lotes": int}
{"comando": "LOTE_ABIERTO", "lote": LoteEnSubasta}
{"comando": "NUEVA_PUJA", "id_lote": int, "usuario": str, "monto": int, "tiempo_restante": float}
{"comando": "LOTE_EXTENDIDO", "id_lote": int, "extension": int, "segundos": int}
{"comando": "LOTE_CERRADO", "id_lote": int, "ganador": str | None, "precio_final": int | None}
{"comando": "REMATE_TERMINADO", "ranking": list[tuple[str, int, int]]}
{"comando": "SERVIDOR_CERRANDO"}
```

6.2. Codificación de bytes

Todo lo que viaja por el `socket`, en ambas direcciones, debe tener exactamente este formato. El archivo `protocolo.py` (idéntico en cliente y servidor; solo puede importar `parametros.py`) debe exponer al menos:

```
def cifrar(datos: bytes) -> bytearray
def descifrar(datos: bytes) -> bytes
def codificar(mensaje: dict) -> bytes
def decodificar(datos: bytes) -> dict
```

Formato de un mensaje. Sea `datos` el resultado de `pickle.dumps(mensaje)` y `largo` su cantidad de bytes. El mensaje que se envía es:

$$[\text{largo}, 4 \text{ B big endian}] + \sum ([n.^{\circ} \text{ de bloque}, 2 \text{ B little endian}] + [24 \text{ B de contenido cifrado}])$$

- **Cifrado, paso A (suma con clave).** La clave son los 4 bytes de `CLAVE_CIFRADO.to_bytes(4, "big")` (para `CLAVE_CIFRADO = 22331103` son 01 54 BE DF). El byte en la posición i se reemplaza por $(\text{datos}[i] + \text{clave}[i \% 4]) \% 256$.
- **Cifrado, paso B (rotación).** Con $k = \text{largo} \% \text{MODULO_ROTACION}$, la secuencia completa se rota k posiciones **a la izquierda**: los primeros k bytes pasan al final.
- **Bloques.** La secuencia cifrada se divide en bloques de `TAMANO_BLOQUE` (24) bytes; el último se rellena con `b"\x00"` hasta completar 24. Cada bloque va precedido por su número, **partiendo desde 1**, en `BYTES_NUMERO_BLOQUE` (2) bytes **little endian**.
- **Header.** Al principio del mensaje va `largo` (el largo del pickle, sin contar relleno) en `BYTES_LARGO` (4) bytes **big endian**.

El mensaje tiene $\lceil \text{largo}/24 \rceil$ bloques y ocupa $4 + 26 \cdot \text{bloques}$ bytes. `descifrar` y `decodificar` son las operaciones inversas exactas: `decodificar(codificar(m)) == m` para todo mensaje.

Lo que la recepción de un mensaje debe cumplir:

- Debe reconstruir el mensaje completo aunque `recv` entregue menos bytes de los pedidos o los entregue en varias partes, y aunque el mensaje sea más grande que `TAMANO_RECV`.
- Debe detectar un mensaje corrupto: un número de bloque fuera de secuencia, un largo inconsistente con lo recibido o un contenido que no se puede deserializar. Detectarlo no cierra la conexión.
- Debe distinguir un mensaje corrupto de una conexión cerrada por la contraparte (`recv` retorna `b""`), que sí termina la conexión.

Ejemplo 1: solo cifrado

`cifrar(b"DCC!!")`: largo = 5, k = 0. Suma byte a byte: $0x44+0x01=0x45$, $0x43+0x54=0x97$, $0x43+0xBE=0x101 \rightarrow 0x01$, $0x21+0xDF=0x100 \rightarrow 0x00$, $0x21+0x01=0x22$. Resultado: 45 97 01 00 22.

Ejemplo 2: mensaje completo

Para `{"comando": "SALIR"}`, con Python 3.12 `pickle.dumps` produce estos 33 bytes:

80 04 95 16 00 00 00 00 00 00 00 7d 94 8c 07 63 6f 6d 61 6e 64 6f 94 8c 05 53 41 4c 49 52 94 73 2e

Paso A ($0x80+0x01=0x81$, $0x04+0x54=0x58$, $0x95+0xBE=0x153 \rightarrow 0x53$, $0x16+0xDF=0xF5$, ...):

81 58 53 f5 01 54 be df 01 54 be 5c 95 e0 c5 42 70 c1 1f 4d 65 c3 52 6b 06 a7 ff 2b 4a a6 52 52 2f

Paso B: $k = 33 \% 5 = 3$; los primeros 3 bytes (81 58 53) pasan al final:

f5 01 54 be df 01 54 be 5c 95 e0 c5 42 70 c1 1f 4d 65 c3 52 6b 06 a7 ff 2b 4a a6 52 52 2f 81 58 53

Bloques: $\lceil 33/24 \rceil = 2$. Header: $33 = 0x21$. Mensaje final (56 bytes):

00 00 00 21

01 00 f5 01 54 be df 01 54 be 5c 95 e0 c5 42 70 c1 1f 4d 65 c3 52 6b 06 a7 ff

02 00 2b 4a a6 52 52 2f 81 58 53 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

Si tu `pickle.dumps` produce bytes distintos (otra versión de Python, otro orden de claves), usa igualmente los 33 bytes de arriba como entrada de `cifrar` para verificar tu implementación.

7. Serialización

7.1. pickle y modelos.py

El archivo `modelos.py` (idéntico en cliente y servidor) define las clases que viajan por la red. Como ambos programas se ejecutan con `python3 main.py` desde su carpeta, `pickle` encuentra las mismas clases en los dos lados. Debe contener:

- Las `namedtuple` `Lote`, `Postor` y `Registro` de la sección 9.
- `class Adjudicacion` con atributos `id_lote`, `nombre_lote`, `usuario`, `precio_final`, `comision` y `fecha` (`datetime`), y un `__str__` legible para el cliente.
- `class LoteEnSubasta`: el estado de un lote mientras está abierto. En el servidor debe conocer, al menos, el `Lote`, el precio actual, el líder, las pujas recibidas, las extensiones usadas y el instante programado de cierre.

Lo que `LoteEnSubasta` debe cumplir al viajar por la red:

- Se debe poder responder `ESTADO_LOTE` sobre un lote que está **activamente siendo subastado**, es decir, con su cierre programado y pendiente, y el cliente debe recibir el objeto correctamente.
- Debe poder enviarse con `pickle` **tal como está en el servidor**, aunque en ese momento contenga atributos que `pickle` no puede serializar (por ejemplo, el temporizador de cierre, si decides guardarlo ahí). No está permitido evitar el problema enviando un diccionario o una copia “limpia” construida a mano: la solución debe estar en la propia clase, personalizando cómo `pickle` la serializa y la reconstruye.
- Lo que recibe el cliente **no** debe incluir las pujas individuales (solo `cantidad_pujas: int`) y **sí** debe incluir `tiempo_restante: float`, calculado en el instante de serializar.

- En el cliente, el objeto reconstruido debe poder mostrarse con la opción “Ver lote en subasta” sin depender de nada que solo exista en el servidor.

7.2. JSON

- `conexion.json` (uno en cliente/ y otro en servidor/; se entregan):

```
{"host": "localhost", "puerto": 8080, "puerto_api": 5000}
```

Si falta el archivo o no es JSON válido, el programa termina con un mensaje claro, no con un traceback. El host y los puertos no pueden estar escritos en el código.

- `servidor/data/remates.json`: lista de `Adjudicacion` de todas las ejecuciones anteriores. La conversión entre objetos `Adjudicacion` (con su `datetime`) y JSON debe hacerse personalizando la serialización de `json` en ambas direcciones (un `JSONEncoder` propio al escribir y un `object_hook` al leer); no se acepta armar y desarmar los diccionarios a mano fuera de ese mecanismo. Si el archivo no existe al iniciar, el servidor lo crea con `[]`. Un lote que aparece en este archivo no se vuelve a subastar. Formato:

```
[
  {
    "__tipo__": "Adjudicacion",
    "id_lote": 1,
    "nombre_lote": "Reloj cucu con olor a cafe",
    "usuario": "tperez",
    "precio_final": 10700,
    "comision": 535,
    "fecha": "2026-10-20T19:04:31.486823"
  }
]
```

8. WebServices

El servidor expone una API Flask de **solo lectura** en `puerto_api` de `conexion.json`, en el mismo proceso que el servidor TCP y disponible durante toda su ejecución. Toda respuesta lleva su código HTTP explícito.

Endpoint	Parámetros	Respuesta 200	Errores
GET /lotes	?categoria=<str> opcional	{"lotes": [{"id_lote", "nombre", "categoria", "anio", "precio_base", "estado"}, ...], "resumen": {"Arte": 94900, ...}}	400 {"error": "categoria_invalida"} si la categoría no es una de las cinco.
GET /lotes/<int:id_lote>		{"id_lote", "nombre", "categoria", "anio", "precio_base", "vendedor", "estado", "precio_actual", "lider"} (los dos últimos null salvo que esté en subasta)	404 {"error": "lote_inexistente"}
GET /ranking	?n=<int> opcio- nal (por defecto TOP_RANKING)	{"ranking": [{"posicion": 1, "usuario": "jsilva", "lotes": 16, "total": 240700}, ...], "por_categoria": {"Arte": 55, ...}}	400 {"error": "n_invalido"} si n no es un entero positivo.

estado es uno de "disponible", "en_subasta", "adjudicado" o "sin_ofertas" (cerrado en esta ejecución sin pujas). El ranking considera `historial.csv` más **todas** las adjudicaciones conocidas (las de `remates.json` y las de esta sesión), y se calcula con `ranking_postores` y `adjudicaciones_por_categoria` de `consultas.py`.

El cliente consume los tres `endpoints` con `requests`, revisa el código de estado y maneja la respuesta de error de cada uno (opciones 4, 5 y 6 del menú).

9. Archivos del programa

9.1. Estructura de la entrega

Los siguientes archivos deben existir con ese nombre y en esa ubicación. Cómo repartes el resto del código en módulos es tu decisión, pero cada archivo debe tener a lo más 400 líneas y **cliente/** y **servidor/** **no pueden importar archivos de la otra carpeta**: trátalos como si corrieran en computadores distintos; los módulos comunes (`protocolo.py`, `modelos.py`) se copian.

```
EP2/
|-- README.md
|-- .gitignore
|-- requirements.txt          (flask y requests)
|-- cliente/
|   |-- main.py
|   |-- protocolo.py          copia idéntica
|   |-- modelos.py            copia idéntica
|   |-- parametros.py
|   '-- conexion.json
'-- servidor/
    |-- main.py
    |-- consultas.py          programación funcional
    |-- protocolo.py          copia idéntica
    |-- modelos.py            copia idéntica
    |-- parametros.py
    |-- conexion.json
    '-- data/
        |-- lotes.csv
        |-- postores.csv
        |-- historial.csv
        '-- remates.json      lo crea el servidor; no se versiona
```

9.2. Archivos CSV

Los tres archivos van en `servidor/data/`, se entregan junto a este enunciado y cumplen las siguientes condiciones:

- Están codificados en UTF-8 y utilizan una coma (,) como separador.
- La primera línea corresponde al encabezado y no representa un elemento del programa.
- Ningún campo contiene comas. Los nombres pueden contener espacios.
- Las categorías son exactamente Arte, Antigüedades, Tecnología, Coleccionables y Rarezas; las regiones, Norte, Centro, Sur e Insular.

9.2.1. lotes.csv

Cada fila representa un lote del catálogo (60 filas):

```
id_lote,nombre,categoria,anio,precio_base,vendedor
1,Reloj cucu con olor a cafe,Antiguedades,1894,10600,Bodega
2,Mural de post-its de edicion limitada,Arte,1912,11300,Secretaria
```

```
Lote(id_lote: int, nombre: str, categoria: str, anio: int, precio_base: int, vendedor: str).
```

9.2.2. postores.csv

Cada fila representa un postor registrado (25 filas). Solo estos usuarios pueden ingresar al remate; **saldo** es el saldo inicial de cada ejecución del servidor:

```
usuario,nombre,region,saldo
rmunoz,Rosa Munoz,Norte,12500
tperez,Tomas Perez,Centro,16000
```

```
Postor(usuario: str, nombre: str, region: str, saldo: int).
```

9.2.3. historial.csv

Cada fila es un remate de “temporadas anteriores” (250 filas). Un mismo lote puede aparecer varias veces (los ganadores lo devolvieron porque no cabía en el departamento). **fecha** se mantiene como **str**:

```
id_remate,id_lote,usuario,precio_final,fecha
1,6,fgutierrez,15100,2024-03-11 18:28:00
2,23,vmorales,12000,2024-03-11 19:07:00
```

```
Registro(id_remate: int, id_lote: int, usuario: str, precio_final: int, fecha: str).
```

9.3. .gitignore

Tu repositorio debe incluir un archivo `.gitignore` en EP2/ que ignore al menos `__pycache__/, *.pyc, .venv/, venv/, .idea/, .vscode/, *.log` y `servidor/data/remates.json`. `remates.json` se genera en ejecución y no debe versionarse; el servidor debe crearlo si no existe.

10. Parámetros

Todos los valores mostrados en [ESTE_FORMATO](#), los nombres de archivos y cualquier otro valor constante deben definirse en `parametros.py` (cada carpeta tiene el suyo; los del protocolo deben ser idénticos en ambos). Se entregan con los siguientes valores, que permiten comenzar a probar el programa:

Parámetro	Valor	Parámetro	Valor
MAXIMO_POSTORES	8	MINIMO_POSTORES	2
LOTES_POR_REMATE	6	DURACION_LOTE	20
UMBRAL_EXTENSION	5	EXTENSION_LOTE	10
MAXIMO_EXTENSIONES	3	TIEMPO_ENTRE_LOTES	5
INCREMENTO_MINIMO	100	COMISION_CASA	0.05
TOP_RANKING	5	INTENTOS_INGRESO	3
TIMEOUT_RESPUESTA	5	TAMANO_RECV	4096
BYTES_LARGO	4	BYTES_NUMERO_BLOQUE	2
TAMANO_BLOQUE	24	MODULO_ROTACION	5
CLAVE_CIFRADO	22331103		

Importante: tu programa debe funcionar con cualquier valor razonable de los parámetros de la subasta; durante la revisión se cambiarán. Los parámetros del protocolo ([BYTES_LARGO](#), [BYTES_NUMERO_BLOQUE](#), [TAMANO_BLOQUE](#), [CLAVE_CIFRADO](#), [MODULO_ROTACION](#)) no deben modificarse, pues el ejemplo de la sección 6.2 depende de ellos.

11. Pruebas

No se entregan tests. Antes de entregar, verifica al menos lo siguiente:

- `decodificar(codificar(m)) == m` para mensajes de distintos tamaños (menores, iguales y mayores a un bloque, y mayores a [TAMANO_RECV](#)), y los dos ejemplos de la sección 6.2.
- Las consultas de `consultas.py` contra los CSV entregados (sección 5.4) y contra archivos modificados por ti.
- Un remate completo con al menos **tres clientes** conectados, con pujas simultáneas por el mismo monto, una puja en los últimos segundos de un lote, un cliente que se cierra con Ctrl+C siendo líder, un cliente que vuelve a ingresar, los tres endpoints con parámetros válidos e inválidos, y el servidor terminando por sí solo.
- Ctrl+C al servidor con clientes conectados.
- Una segunda ejecución del servidor, para comprobar que no se resubastan los lotes de `remates.json`.
- Todo lo anterior con otros valores de los parámetros de la subasta.

12. Restricciones y alcances

- Este ejercicio es **estrictamente individual** y está regido por el [Código de Honor](#) de Ingeniería.
- Tu programa debe ser desarrollado utilizando Python **3.12.X**, con X mayor o igual a 0.
- Tu programa debe estar compuesto por uno o más archivos con extensión `.py`, organizados coherentemente dentro de la carpeta de entrega. **No se revisarán archivos en otros formatos**, como `.ipynb`.
- El programa será ejecutado **únicamente desde la terminal**, ubicando la terminal dentro de `servidor/` o de `cliente/` según corresponda y utilizando el siguiente comando:

```
python3 main.py
```

- Durante la revisión se podrá cambiar el nombre de la carpeta que contiene el **Ejercicio Propuesto** por otro nombre. Por lo tanto, los paths relativos utilizados en el programa deben ser coherentes.
- Solo se revisarán entregables que puedan ejecutarse correctamente desde la consola. Si el programa no puede iniciarse, se cierra inesperadamente antes de acceder a sus principales funcionalidades o requiere pasos no documentados, el entregable podrá no ser revisado y no se entregará feedback.
- La consola del cliente debe ser robusta frente a opciones inválidas, texto en campos numéricos e identificadores inexistentes; el servidor debe ser robusto frente a desconexiones y mensajes corruptos. Se permite utilizar `try/except` para validar conversiones, operaciones de archivos y de red, **especificando siempre la excepción que se captura**. No se acepta `except:` ni `except Exception:`.
- Se espera que el código sea legible, esté organizado coherentemente y cumpla con las buenas prácticas de programación trabajadas en el curso. En particular, se espera:
 - Utilizar nombres descriptivos para variables, funciones, clases y módulos.
 - Distribuir las responsabilidades del programa de manera coherente.
 - Evitar la repetición innecesaria de código.
 - Evitar concentrar toda la solución en un único archivo, función o componente; ningún archivo debería superar las 400 líneas.
 - Mantener un formato consistente que facilite la lectura del código (PEP8, líneas de hasta 100 caracteres).
 - Documentar las decisiones que no sean evidentes.

Si el código no presenta condiciones mínimas de legibilidad u organización que permitan comprender su funcionamiento, el entregable podrá no ser revisado y no se entregará feedback.

- Debe existir una separación clara entre la interacción por consola, la lógica de la subasta, las consultas funcionales, el protocolo y el manejo de archivos. Puedes distribuir el código en los módulos que consideres apropiados, respetando los nombres obligatorios de la sección 9.
- Solo se permiten módulos de la biblioteca estándar de Python (en particular `socket`, `threading`, `queue`, `json`, `pickle`, `functools`, `itertools`, `collections`, `datetime`, `math`, `os`, `sys` y `time`) más `flask` y `requests`. Si no se encuentra especificado en el enunciado, supón su uso **está prohibido**. Pregunta en una discussion en el **foro** si es que es posible utilizar alguna librería en particular.
- Este ejercicio debe desarrollarse **exclusivamente con los contenidos liberados** como materia regular al momento de publicar el enunciado.
- Está prohibido citar o utilizar código publicado **después de la liberación del enunciado**. Solo se permite citar contenido que ya se encontrara disponible antes de su publicación, referenciándolo en el `README.md`.
- Ante cualquier duda relacionada con el enunciado, puedes utilizar Discussions.
- Cualquier aspecto no especificado queda a tu criterio, siempre que no contradiga una regla explícita del enunciado. Los supuestos realizados deben aplicarse consistentemente y quedar explicados en el archivo `README.md`, junto con las instrucciones de ejecución, el estado de la entrega y una explicación breve de cómo organizaste los threads y qué recursos protegiste.

Los entregables que no cumplan con las restricciones de ejecución, formato o legibilidad podrán no ser revisados y no recibir feedback.